Mutchumari Chandrsekhar Reddy

AP25110010976

1. Greedy Approach: Implementation of Huffman code

Program:

```
#include<stdio.h>
#include<stdlib.h>
struct Node{
int data;
char ch;
struct Node *left,*right;
};
struct Node* newNode(int data,char ch){
struct Node* node=(struct Node*)malloc(sizeof(struct Node));
node->data=data;
node->ch=ch;
node->left=node->right=NULL;
return node;
}
void sort(struct Node* a[],int n){
int i,j;
struct Node* temp;
for(i=0;i<n-1;i++)
for(j=0;j<n-i-1;j++)
if(a[j]->data>a[j+1]->data){
temp=a[j];
a[j]=a[j+1];
a[j+1]=temp;
}
}
void printCodes(struct Node* root,int code[],int top){
int i;
if(root->left){
code[top]=0;
printCodes(root->left,code,top+1);
}
if(root->right){
```

```
code[top]=1;
printCodes(root->right,code,top+1);
}
if(!root->left&&!root->right){
printf("%c: ",root->ch);
for(i=0;i<top;i++)
printf("%d",code[i]);
printf("\n");
}
}
int main(){
int n,i,code[100];
char ch;
struct Node* a[100];
printf("Enter number of characters: ");
scanf("%d",&n);
printf("Enter character and frequency:\n");
for(i=0;i<n;i++){
scanf(" %c%d",&ch,&code[i]);
a[i]=newNode(code[i],ch);
}
while(n>1){
sort(a,n);
struct Node* left=a[0];
struct Node* right=a[1];
struct Node* parent=newNode(left->data+right->data,'$');
parent->left=left;
parent->right=right;
a[0]=parent;
for(i=1;i<n-1;i++)
a[i]=a[i+1];
n--;
}
printf("Huffman Codes:\n");
printCodes(a[0],code,0);
return 0;
}
```

Terminal Process:

```
PS C:\Users\chand\OneDrive\DAA-2026> cd "c:\Users\chand\OneDrive\DAA-2026\" ; if ($?) { gcc huffman.c -o huffman } ; if ($?) { .\huffman }
Enter number of characters: 4
Enter character and frequency:
a 5
b 9
c 12
d 13
Huffman Codes:
a: 00
b: 01
c: 10
d: 11
PS C:\Users\chand\OneDrive\DAA-2026>
```

2. Assume that the numbers given below represent counts of letters in the hundreds from a file (similar to the CLRS example). For example, in the file there will be exactly 20

- 100 occurrences of the letter ‘a’ , 11*100 occurrence of the letter ‘c’, etc. a: 20, c:11, d:2, e: 10, o:15, m:8, s:10, t:22, u: 2 a) What is an optimal Huffman code based on the following set of frequencies?

1. Draw the tree. Show your work at every step.
2. Fill in the table on the right the Huffman encoding for each letter.
3. We encode the file using the Huffman codes produced above. How much memory will the file require with this encoding?

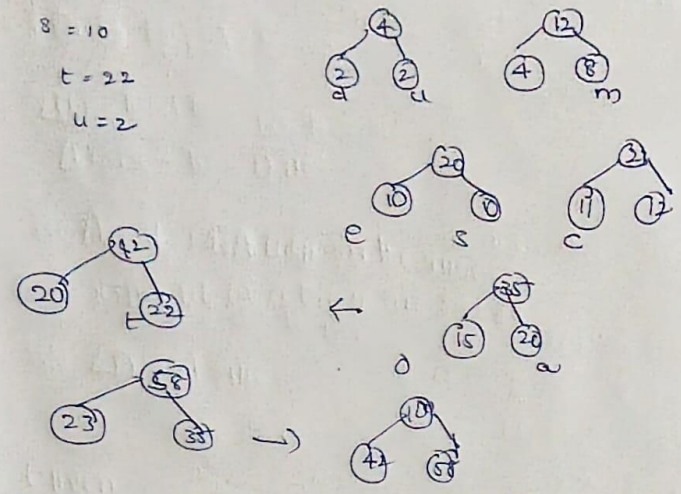
Handwritten Solution:

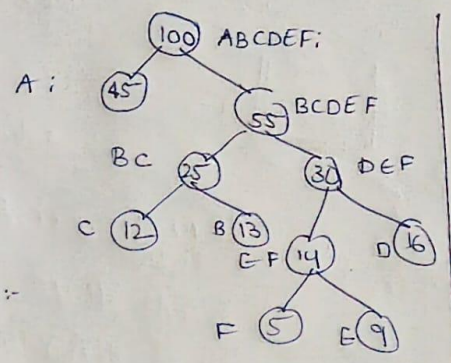
(2)

a = 20
c = 11
d = 2
e = 10
o = 15
m = 8
s = 10
t = 22
u = 2

Huffmann tree

2 2 4 8 10 10 11 12
18 20 20 22
23 35
42 58





Memory :-

Node

a = 111 → 20 → 3 → 60
c = 100 → 11 → 3 → 33
d = 10100 → 2 → 5 → 10
e = 000 → 10 → 3 → 30
o = 110 → 15 → 3 = 45
m = 1011 → 8 → 4 = 32
s → 001 → 10 → 3 = 30
t → 01 → 22 → 22 = 44
u → 10101 → 2 → 5 = 10

100

160 Freq Total mem = 294 bits
In bytes 294/8 = 36.75
37 bytes